

Cursul 11 - Java Servlet Technology

Structura cursului

- Noțiuni introductive
- Conținut Static și Dinamic
- Common Gateway Interface (CGI)
- Lucrul cu Servlets și clasa `Servlet`
- Exemple de implementare

Introducere

Servlet-ii reprezintă aplicații destinate rulării pe servere și presupun tehnologia Java Servlet ce conține suportul necesar pentru el (API-uri, containere destinate dezvoltării resurselor necesar rulării servlet-ilor).

Servlet-ii se diferențiază de aplicațiile Java Standard prin faptul că rulează pe servere și necesită impunerea unui nivel ridicat de securitate.

Principala clasă pentru utilizarea servlet-ilor este clasa `Servlet`.

Conținut Static și Dinamic

Din perspectiva conținutului static, servlet-ii au doar rolul de a afișa informații, pe când, la conținutul dinamic, lucrurile sunt cu totul diferite:

- trebuie preluată cererea de la client
- construirea codului specific cererii
- procesării resurselor de pe server
- construirea unui răspuns favorabil

Tehnologia servlet actuală permite programatorilor să construiască un răspuns eficient, cu un nivel ridicat de securitate și cu posibilitatea tratării cererilor multiple.

Modul de lucru cu servlet-urile

Cererea și răspunsul HTTPS (HyperText Transfer Protocol Secure) reprezintă un protocol folosit de către World Wide Web. În fiecare cerere HTTPS este specificată:

- Metoda GET - folosită, în principal, pentru a primi informații de la server, cererea fiind transmisă sub forma unui URL (Uniform Resource Locator)
- Metoda POST - folosită pentru a trimite date către server, datele fiind încapsulate într-un pachet, pachetul fiind transmis server-ului

Clasele și interfețele necesare pentru dezvoltarea componentelor servlet se găsesc în

pachetele `javax.servlet` și `javax.servlet.http`.

Orice servlet trebuie să implementeze interfața `Servlet`, fie direct, fie prin extinderea clasei `HttpServlet`.

Un servlet poate fi instalat (deployed) în orice container de servlets, precum web server-ul TomCat sau server-ul de aplicații Java Enterprise.

OBS: Pentru instalare, sunt prevăzute anumite convenții de stocare a claselor care sunt respectate de toate containerele web, dar mai există și unele aspecte particulare fiecărui container. De exemplu, server-ul TomCat ascultă cererile de la clienți pe portul 8080, și pentru fiecare aplicație web, se definește un subdirector de bază (document root) în directorul web apps al instalării.

Structura unui servlet

Pentru a fi un servlet, o clasă trebuie să moștenească `HttpServlet` și să suprascriere metoda `doGet()` sau `doPost()` sau ambele. Metodele `doGet()` și `doPost()` au două argumente:

- `HttpServletRequest` ce are metode care permit primirea datelor
- `HttpServletResponse` ce are metode cu care specifică tipul de răspuns pe care îl dăm și mai ales permite obținerea unui obiect de tip `PrintWriter` pe care îl folosim pentru a trimite răspunsuri pentru clienți.

Structura generală al unui servlet este:

```
package pkg;

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Servlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res)
    throws IOException, ServletException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<html><head><title>Sample Servlet");
        out.println("</title></head><body>");
        out.println("<h1>Hello, world at " +
                    req.getRequestURI() + "!</h1>");
        out.println("<p>Key is " + req.getParameter("KEY"));
        out.println("</p></body></html>");
    }
}
```

Metode uzuale în lucrul cu servlets:

- `init(config)`

- Service:
 - `doGet(req, res)`
 - `doPut(req, res)`
 - `doDelete(req, res)`
- `getParameter(name)`
- `getCookies()`
- `sendRedirect(url)`
- `sendError(errCode, errStr)`
- `addCookie(cookieObj)`